Experiment 'p_6_6_alternating_sum_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_6_alternating_sum_array
Natural language statement: null
Method signature: p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)

Ensures

- `alt_sum == sum_array(inputs[..], 1)`

Functional Code Given

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)

The method should respect the following contract:

ensures alt_sum == sum_array(inputs[..], 1)

The contract uses the following dafny code:

function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnOMXYe3jvgpfazOq83ENUg27C0mG
Model: gpt-5.5-2026-04-23
Created at: 1780664093
Finish reason: stop
Usage: 410 tokens in, and 4540 tokens out

```
{
  alt_sum := 0;
  var idx := 0;
  var sign := 1;

  assert inputs[0..] == inputs[..];
  assert alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1);

  while idx < inputs.Length
    invariant 0 <= idx <= inputs.Length
    invariant alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1)
```

```
    decreases inputs.Length - idx
  {
    var oldIdx := idx;
    var oldSign := sign;
    var rest := inputs[oldIdx..];

    assert 0 <= oldIdx < inputs.Length;
    assert |rest| > 0;
    assert rest[0] == inputs[oldIdx];
    assert rest[1..] == inputs[oldIdx + 1..];
    assert sum_array(rest, oldSign) == rest[0] * oldSign + sum_array(rest[1..], oldSign *
        ↪ -1);
    assert sum_array(inputs[oldIdx..], oldSign) == inputs[oldIdx] * oldSign + sum_array(
        ↪ inputs[oldIdx + 1..], oldSign * -1);

    alt_sum := alt_sum + inputs[oldIdx] * oldSign;
    sign := oldSign * -1;
    idx := oldIdx + 1;

    assert alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1);
  }

  assert idx == inputs.Length;
  assert inputs[idx..] == [];
  assert sum_array(inputs[idx..], sign) == 0;
  assert alt_sum == sum_array(inputs[..], 1);
}
```

Final Program

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}

method p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)
       ensures alt_sum == sum_array(inputs[..], 1)
{
  alt_sum := 0;
  var idx := 0;
  var sign := 1;

  assert inputs[0..] == inputs[..];
  assert alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1);

  while idx < inputs.Length
    invariant 0 <= idx <= inputs.Length
    invariant alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1)
    decreases inputs.Length - idx
```

```
  {
    var oldIdx := idx;
    var oldSign := sign;
    var rest := inputs[oldIdx..];

    assert 0 <= oldIdx < inputs.Length;
    assert |rest| > 0;
    assert rest[0] == inputs[oldIdx];
    assert rest[1..] == inputs[oldIdx + 1..];
    assert sum_array(rest, oldSign) == rest[0] * oldSign + sum_array(rest[1..], oldSign *
        ↪ -1);
    assert sum_array(inputs[oldIdx..], oldSign) == inputs[oldIdx] * oldSign + sum_array(
        ↪ inputs[oldIdx + 1..], oldSign * -1);

    alt_sum := alt_sum + inputs[oldIdx] * oldSign;
    sign := oldSign * -1;
    idx := oldIdx + 1;

    assert alt_sum + sum_array(inputs[idx..], sign) == sum_array(inputs[..], 1);
  }

  assert idx == inputs.Length;
  assert inputs[idx..] == [];
  assert sum_array(inputs[idx..], sign) == 0;
  assert alt_sum == sum_array(inputs[..], 1);
}
```

Total Token Usage

Input tokens: 410
Output tokens: 4540
Reasoning tokens: 4095
Sum of 'total tokens': 4950

Experiment Timings

Overall Experiment started at 1780664092979, ended at 1780664207277, lasting 114298ms (114.30 seconds)
Iteration #1 started at 1780664092979, ended at 1780664207277, lasting 114298ms (114.30 seconds)